

ML Fundamentals & Libraries

Data Science / AI Fundamentals

Supervised vs. Unsupervised Learning

Supervised learning trains a model on labeled data — input examples paired with the correct answer — so the model learns to predict that answer for new, unseen inputs. It splits further into classification (predicting a category, like spam/not-spam) and regression (predicting a continuous number, like a house price). Unsupervised learning works with unlabeled data, finding structure without being told the 'right answer' — clustering (grouping similar data points together) and dimensionality reduction (compressing many features into fewer, more informative ones) are the two most common tasks. The choice between them starts with one question: does your data have labeled correct answers to learn from, or not?

Model Evaluation Basics

A model that performs well on the data it was trained on tells you almost nothing about how it will perform on new data — this is why data is split into a training set (used to fit the model) and a test set (held out, used only to evaluate it afterward). Overfitting happens when a model learns the training data's noise and quirks rather than its genuine underlying pattern, performing great on training data but poorly on new data; underfitting is the opposite — a model too simple to capture the real pattern, performing poorly on both. For classification, accuracy alone can be misleading on imbalanced data (e.g. 99% accuracy predicting 'not fraud' when only 1% of transactions are actually fraud is worthless) — precision, recall, and the confusion matrix give a fuller picture.

NumPy & Pandas Workflows

NumPy provides fast, memory-efficient arrays and the mathematical operations that work on them (this is the foundation almost every other data/ML library in Python is built on top of). Pandas builds on NumPy to provide the DataFrame — a labeled, tabular structure much like a spreadsheet or SQL table — along with operations to filter, group, join, and reshape data. A typical real workflow moves from raw files, through Pandas for cleaning and exploring, into NumPy arrays (often implicitly, via scikit-learn) for the actual model training step.

scikit-learn Pipelines

scikit-learn provides a consistent interface across dozens of ML algorithms — nearly every model exposes the same `.fit(X, y)` (train on data) and `.predict(X)` (make predictions) methods, which makes it straightforward to try multiple algorithms on the same problem with minimal code changes. A Pipeline chains together preprocessing steps (like scaling numeric features or encoding categorical ones) and a final model into a single object, ensuring the exact same

preprocessing is applied consistently to both training data and any new data the model later predicts on — a common source of subtle bugs when preprocessing is done manually and inconsistently.

Learn More

scikit-learn Official Documentation

<https://scikit-learn.org/stable/>

NumPy Official Documentation

<https://numpy.org/doc/>

Google's Machine Learning Crash Course

<https://developers.google.com/machine-learning/crash-course>